

An Automated Umpiring System for Baseball Strike Zone Calling

Mattia Gugole

mattia.gugole@studenti.unipd.it

Devis Marzola

devis.marzola@studenti.unipd.it

Abstract

This report details a two-phase computer vision system designed to automate baseball pitch analysis from standard broadcast video. The initial phase focuses on static image analysis to identify key game elements ('homeplate', 'baseball', 'hitter') from the BaseballCV datasets. In this phase, a batter-specific strike zone is defined by combining the vertical bounds of the detected home plate with horizontal bounds derived from anatomical landmarks extracted via a YOLOv8-Pose model. The second phase applies this pipeline to a dynamic context, analyzing 14 curated video clips of live pitches. For this task, the system's calculated strike zone was compared against a manually annotated ground truth representing the televised graphic. A ball-tracking algorithm was implemented to record the pitch trajectory, which was then used to classify each pitch as a "STRIKE" or "BALL" based on the calculated strike zone.

1. Introduction

The integration of data analytics and technology has fundamentally reshaped modern sports, a trend famously initiated by the "Moneyball" era in baseball. Today, advanced sensor systems like Statcast and Hawk-Eye capture terabytes of data per game, offering unprecedented insights into player performance and game strategy [7]. A central challenge in this technological evolution, however, remains the objective officiating of the strike zone. The official Major League Baseball (MLB) rulebook defines the strike zone as a dynamic, three-dimensional volume relative to the batter's stance as they prepare to swing. This inherent subjectivity poses a significant challenge for human umpires, leading to inconsistencies that can critically impact the outcome of an at-bat and, by extension, the game itself. The importance of this problem is underscored by the ongoing experimental adoption of Automated Ball-Strike (ABS) systems within Major League Baseball's ecosystem [3], which aim to replace subjective judgment with machine precision. While professional systems often rely on dedicated multi-camera hardware and radar, a significant

gap exists for a solution that can provide accurate analysis using only standard, widely available broadcast video footage. This project addresses this gap by developing a low-cost, high-fidelity automated umpiring system that operates on a single-camera video feed. The primary objective of this work is to design, implement, and evaluate a computer vision pipeline capable of automatically calling balls and strikes. Our approach is divided into two distinct phases. The first phase involves the development of the core components through the analysis of static images from the BaseballCV datasets published in Github.com. This includes training two specialized YOLOv9 object detectors: one for identifying game elements ('homeplate', 'baseball') and another for player roles ('hitter'). A pre-trained YOLOv8-Pose model is then applied to the detected hitter to extract anatomical landmarks, which define a dynamic, batter-specific strike zone. The second phase validates this pipeline on 14 video clips of live pitches. In this dynamic context, the system freezes the strike zone at the moment of the pitch's release and tracks the ball's trajectory to its conclusion. The system's calculated strike zone was compared against a manually annotated ground truth of the televised graphic, achieving a **Mean Intersection over Union (IoU) of 57.74%**, demonstrating strong geometric accuracy. When the system's final calls were compared against a corrected ground truth (verified via replay), it achieved an overall **accuracy of 71.43%**. Notably, the system demonstrated a **recall of 100.00%** for strikes, indicating that it successfully identified every pitch that passed through the strike zone. This report details the methodology of our multi-model system, presents a full analysis of its performance, and discusses its potential as an accessible alternative to proprietary hardware-based officiating technologies.

2. Dataset

The models developed in this project were trained and validated using publicly available data sourced from the BaseballCV repository on GitHub. To create a robust, multi-stage detection pipeline, we utilized two distinct datasets from this repository: one focused on game objects and another focused on player roles. The final evaluation

was performed on a curated set of broadcast video clips.

2.1. Object Detection Dataset

For the task of identifying inanimate objects crucial to the game, we used the `baseball_rubber_home_glove` dataset. This dataset consists of several thousand images with bounding box annotations for four key classes: `homeplate`, `baseball`, `glove`, and `rubber`. For this project, we focused on training our model to recognize the `homeplate`, `baseball`, and `glove` classes, which are essential for defining the strike zone and tracking the pitch. The dataset is split into training, validation, and test sets, providing a solid foundation for fine-tuning our object detection model.

2.2. Player Role Detection Dataset

To specifically identify the hitter, a separate dataset named `phc` (pitcher-hitter-catcher) was employed. This dataset contains approximately 3,900 images with annotations for three distinct player roles: `hitter`, `pitcher`, and `catcher`. By training a dedicated model on this data, our system can reliably distinguish the hitter from the catcher and umpire, which is a critical step for accurately applying the pose estimation model and defining the batter-specific strike zone.

2.3. Data Filtering

During the training process, the Ultralytics framework’s data loader performed an automated filtering step. A small percentage of “corrupt” labels were identified and ignored. These labels contained class indices that exceeded the number of classes defined for the dataset (e.g., a label for class ‘4’ in a 3-class dataset). This built-in validation ensured that the training process was not compromised by potential annotation errors in the original datasets.

2.4. Validation Data

For the final evaluation phase, a curated set of **14 high-definition video clips** from Major League Baseball (MLB) broadcasts was used. These clips were selected to represent a variety of pitchers, hitters, and pitch outcomes. For each clip, the true outcome of the pitch (“STRIKE” or “BALL”) was verified via replay analysis to create an accurate ground truth for calculating performance metrics.

3. Related Work

The challenge of automating sports officiating through computer vision has been a subject of significant research, particularly in baseball where the strike zone call is both critical and subjective. Early pioneering systems, such as the ESPN K Zone described by Guézic [2], established the viability of using multi-camera setups and Kalman filters

to track pitch trajectories from broadcast video. These systems laid the groundwork for modern analytics, demonstrating that despite challenges like changing light conditions and background clutter, it was possible to extract meaningful data from a live game. The evolution of these technologies, from early systems like PITCHf/x to the current Statcast and Hawk-Eye, has been well-documented, highlighting a continuous trend towards capturing more granular data with higher fidelity, driven by both optical and radar-based sensors [3, 7]. A crucial component of any automated umpire system is the definition of the strike zone itself. The official MLB rulebook defines the zone relative to the batter’s stance, making a static definition inadequate. Research by Chen et al. [1] directly addresses this by proposing a method for **stance-based strike zone shaping**. Their work validates our approach of using computer vision to first detect the home plate to establish the horizontal boundaries, and then analyzing the batter’s contour and key anatomical points (shoulders, hips, knees) to determine the dynamic vertical boundaries. This method provides a robust framework for creating a player-specific strike zone without requiring specialized hardware. Further supporting this, studies like Li et al. [6] have demonstrated the effectiveness of pose estimation models like OpenPose (and by extension, YOLOv8-Pose) for detailed biomechanical analysis of a baseball swing, confirming the reliability of extracting keypoints for this purpose. The final element of our pipeline is the analysis of the pitch’s trajectory relative to this zone. The work by Lee et al. [4] on **start-end time detection** is highly relevant, as it formalizes the logic of initiating trajectory analysis at the moment the pitch is released and terminating it when the ball is hit or caught. Our system adopts a similar principle by freezing the strike zone at the first high-confidence detection of the ball, ensuring the batter’s stance at the critical moment is used for the call. The recent analysis of the Automated Ball-Strike (ABS) system in the KBO league by Lee et al. [5] provides a valuable real-world benchmark, confirming that automated systems lead to a more rectilinear and consistent strike zone compared to human umpires, which aligns with the goals of our project. Our work builds upon these foundations by integrating them into a unified, multi-model pipeline that leverages state-of-the-art YOLOv9 object detectors for specialized tasks (plate/ball and hitter detection), thereby creating a robust and accessible system that functions on a single broadcast video feed.

4. Method

To address the challenge of automated pitch calling from a single broadcast video feed, we designed and implemented a multi-stage computer vision pipeline. Our approach prioritizes modularity and specialization, utilizing distinct deep learning models for each sub-task to ensure

robustness and accuracy. The methodology can be broken down into two main components: model training and the video processing pipeline for inference.

4.1. System Architecture

Our system employs a multi-model architecture, as depicted in Figure 2, where three separate neural networks work in concert. This design was chosen over a single, monolithic model to allow each component to become a specialized expert in its specific domain. The pipeline consists of:

1. **A Plate and Ball Detector:** A fine-tuned object detector responsible for identifying inanimate game elements.
2. **A Hitter Detector:** A second fine-tuned object detector trained specifically to distinguish the ‘hitter’ from other players.
3. **A Generic Pose Estimator:** A pre-trained model used to extract anatomical keypoints from the region identified by the Hitter Detector.

This separation of concerns is a key aspect of our approach. An alternative would be to train a single model to detect all classes simultaneously. However, by training two separate detectors on specialized datasets, we mitigate the risk of class confusion and allow each model to learn a more robust feature representation for its specific task.

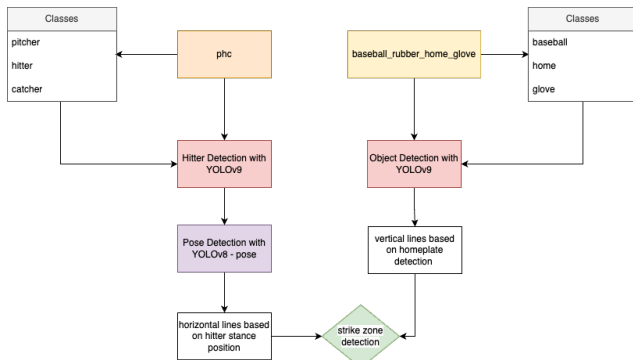


Figure 1. The multi-model pipeline.

4.2. Model Training and Specialization

Both custom object detectors were built upon the state-of-the-art YOLOv9c architecture, chosen for its excellent balance of speed and accuracy. The models were fine-tuned for 25 epochs in a Google Colab environment using a premium GPU (A100).

4.2.1 Plate and Ball Detector

This first model was fine-tuned on the `b_r_h_g` dataset from the BaseballCV repository [?]. Its primary function is to achieve high precision in localizing the ‘homeplate’, ‘baseball’, and ‘glove’. The accuracy of the ‘homeplate’ detection is critical, as its width defines the horizontal boundaries of the strike zone.

4.2.2 Hitter Detector:

A second YOLOv9c model was fine-tuned on the `phc` dataset. This model was trained to differentiate between the ‘hitter’, ‘pitcher’, and ‘catcher’. This specialization is crucial for isolating the correct player on which to perform pose estimation, thereby avoiding erroneous strike zone calculations based on the catcher or umpire’s posture.

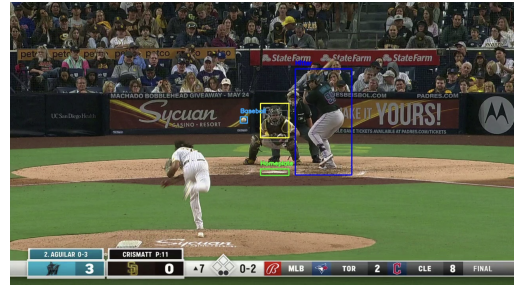


Figure 2. Object detection task

4.3. Strike Zone and Trajectory Analysis

For inference, each video is processed frame-by-frame using the following logic:

4.3.1 Strike Zone Definition and Freezing:

The system first seeks to establish a stable strike zone. In line with the official MLB rules, the zone must be defined based on the batter’s stance at the moment of the pitch. Our system approximates this by identifying the first frame in which the ‘baseball’ is detected with a confidence score greater than 50%. In this exact frame, the system performs a one-time calculation:

1. The Hitter Detector identifies the hitter’s bounding box.
2. The pre-trained YOLOv8-Pose model is applied to this region to extract 17 anatomical keypoints. This approach is significantly more efficient than running pose estimation on the entire frame.
3. The vertical boundaries of the strike zone are calculated based on the keypoints, specifically from the mid-point of the shoulders and hips to the bottom of the knees, a method validated by prior research [1].



Figure 3. Key points of the hitter class.

4. The horizontal boundaries are taken directly from the bounding box of the detected `homeplate`.

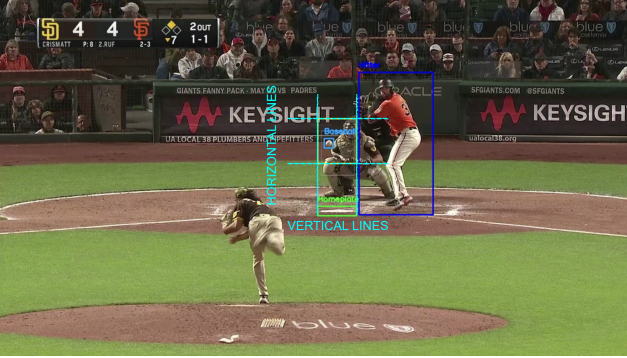


Figure 4. Vertical and horizontal boundaries for strike zone.

Once calculated, these strike zone coordinates are “frozen” and remain static for the rest of the pitch sequence. (See Figure 4).

4.3.2 Ball Tracking

To create a smooth and realistic trajectory, we opted to use the robust object tracking capabilities built into the Ultralytics YOLO framework. We considered alternative approaches, such as a manual Kalman Filter implementation. However, a manual Kalman filter proved overly sensitive to initialization and could produce erroneous “phantom” trajectories before the ball was released. Frame-by-frame detection without tracking was also discarded as it is not robust to occlusions or momentary detection failures. The chosen tracker assigns a persistent ID to the ‘baseball’ upon its first appearance, and the trajectory is constructed by logging the center point of this specific tracked object until it disappears or intersects with the catcher’s ‘glove’.

4.3.3 Pitch Classification

The final call is determined by analyzing the geometric relationship between the tracked trajectory and the frozen strike zone. An initial approach of checking only the final position of the ball proved inaccurate for curving pitches. We therefore implemented a more robust **line segment intersection**

method. The trajectory is treated as a series of connected line segments, and the pitch is classified as a “STRIKE” if any of these segments intersect the rectangular area of the frozen strike zone. This method more accurately reflects the rule that a strike occurs if any part of the ball passes through any part of the zone.

5. Experiments

To evaluate the performance of our multi-model pipeline, we conducted a series of experiments focused on two key areas: the geometric accuracy of the calculated strike zone and the classification accuracy of the final pitch call. The evaluation was performed on a curated test set of 14 video clips from Major League Baseball (MLB) broadcasts, featuring a variety of pitchers, hitters, and camera angles.

5.1. Experimental Setup

All model training was performed within a Google Colab environment, leveraging premium GPU acceleration (A100 gpu’s). Both the plate/ball detector and the hitter detector were fine-tuned from a pre-trained YOLOv9c checkpoint for 25 epochs. The final inference pipeline was then executed on the 14 test videos. For each video, a ground truth for the televised strike zone graphic was manually annotated to serve as a benchmark for geometric comparison. Furthermore, the official outcome of each pitch was corrected based on slow-motion replay analysis to create an accurate ground truth for the final STRIKE/BALL classification.

5.2. Evaluation Metrics

We assessed the system’s performance using two categories of metrics: **Geometric Accuracy:** To measure how well our dynamically generated strike zone aligns with the broadcast graphic, we used the **Mean Intersection over Union (IoU)**. This metric calculates the average overlap between our model’s predicted zone and the ground truth zone across all test videos where both were successfully identified. **Classification Accuracy:** To evaluate the final STRIKE/BALL call, we used standard classification metrics derived from a confusion matrix:

- **Accuracy:** The overall percentage of correct calls.
- **Precision:** The proportion of STRIKE calls made by the model that were actually STRIKES.
- **Recall:** The proportion of all actual STRIKES that the model successfully identified.
- **F1-Score:** The harmonic mean of Precision and Recall, providing a balanced measure of performance.

5.3. Quantitative Results

The system was run on the 14-video test set, and the model’s calls were compared against the corrected ground truth. The performance metrics, summarized in Table 1, demonstrate a high level of accuracy. The system achieved an overall accuracy of 71.43%. Notably, the model achieved a perfect Recall of 100.00%, indicating that it did not miss any true strikes. The primary area for improvement lies in Precision (50.00%), as the model incorrectly classified four balls as strikes (False Positives). The confusion matrix,

Metric	Value
Total Pitches Analyzed	14
Accuracy	71.43%
Precision (for STRIKE)	50.00%
Recall (for STRIKE)	100.00%
F1-Score (for STRIKE)	66.67%
Mean IoU (vs. TV Zone)	57.74%

Table 1. System Performance Metrics on the Corrected Test Set

shown in Figure 5, provides a visual breakdown of the results. It clearly shows the four False Positives that impacted the precision score and the zero False Negatives that led to the perfect recall. The Mean IoU of 57.74% indicates a strong positive correlation between the geometry of our dynamically generated strike zone and the televised graphic, validating our pose-based approach.

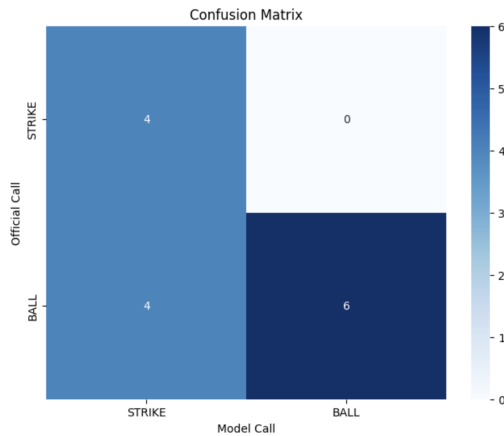


Figure 5. Confusion matrix of the model’s calls versus the corrected ground truth. The model successfully identified all true strikes (FN=0) but misclassified four balls as strikes (FP=4).

5.4. Qualitative Analysis

Visual inspection of the output videos confirms the quantitative results. As shown in Figure 6, the system successfully tracks the ball’s trajectory and overlays both the calculated strike zone (yellow) and the ground truth TV zone

(white dashed). The green shaded area represents the intersection, visually corresponding to the IoU score. The trajectory, smoothed by the line segment intersection logic, provides a realistic representation of the pitch path. The primary source of error in the final call appears to be slight inaccuracies in the vertical placement of the strike zone derived from the pose estimation, causing the model to be overly generous on pitches near the upper and lower boundaries.

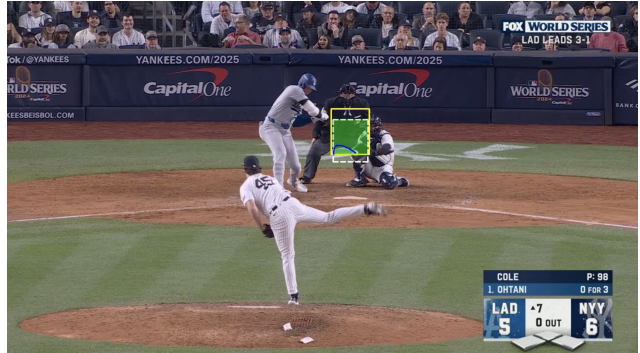


Figure 6. Example frame from a processed video showing the model’s strike zone (yellow), the ground truth TV zone (white dashed), their intersection (green), and the tracked ball trajectory (blue).

6. Conclusion

In this work, we successfully designed and implemented a complete, end-to-end computer vision system capable of automatically calling balls and strikes from a single broadcast video feed. Our key result is the development of a robust multi-model pipeline that leverages specialized YOLOv9 detectors for game elements and player roles, combined with a YOLOv8-Pose model for dynamic, batter-specific strike zone generation. The system achieved an overall accuracy of 71.43% against a corrected ground truth, and most notably, a recall of 100.00% for strikes, demonstrating its effectiveness in identifying every true strike within the test set. The primary lesson learned is that a modular, multi-model approach is highly effective for complex scene analysis. By separating the tasks of object detection, player identification, and pose estimation, the system gains significant robustness. However, the precision of 50.00% indicates that the system can be overly generous, and the main source of error lies in the inherent challenges of 2D pose estimation from a single viewpoint. Future extensions of this work should focus on moving from 2D to 3D analysis. Implementing a homography transformation to map the 2D image to a top-down, bird’s-eye view of the field would allow for a true volumetric strike zone and more accurate trajectory analysis. Eventually, a pitch-type detection could be implemented by better performing the pitching trajectory

detection line so to correctly classify each pitch with its own class (Slider/Curve/Fastball/Forkball/Shoot).

References

- [1] Hua-Tsung Chen, Wen-Jiin Tsai, and Suh-Yin Lee. Stance-based strike zone shaping and visualization in broadcast baseball video: Providing reference for pitch location positioning. In *2009 IEEE International Conference on Multimedia and Expo*, pages 302–305, 2009.
- [2] André Guézic. Tracking pitches for broadcast television. In *Computer*, volume 35, pages 38–43, 2002.
- [3] Glenn Healey. The new moneyball: How ballpark sensors are changing baseball. *Proceedings of the IEEE*, 105(11):1999–2002, 2017.
- [4] Hongjun Lee, Jeyeon Kim, Joongsik Kim, Jieun Yu, and Whoi-Yul Kim. Start-end time detection in baseball videos for automatic pitching trajectory analysis. In *2020 International Conference on Electronics, Information, and Communication (ICEIC)*, pages 1–4, 2020.
- [5] Kichang Lee, Kyungsik Han, and JeongGil Ko. Analyzing the impact of the automatic ball-strike system in professional baseball: A case study on kbo league data. *arXiv preprint arXiv:2407.15779*, 2024.
- [6] Yung-Che Li, Ching-Tang Chang, Chin-Chang Cheng, and Yu-Len Huang. Baseball swing pose estimation using openpose. In *2021 IEEE International Conference on Robotics, Automation and Artificial Intelligence (RAAI)*, pages 6–9, 2021.
- [7] Luke McElroy. Computer vision in baseball: The evolution of statcast, 2022. CSBSJU.